RSBagel Preserves the BAGEL2 Scientific Contract while Accelerating Pooled CRISPR Fitness-Screen Analysis

Yoon Sing Yap

Independent Researcher

Corresponding author: Yoon Sing Yap
yoonatworkz@gmail.com

Abstract

Pooled CRISPR fitness screens identify essential genes by quantifying guide depletion during cell growth. Statistical classifiers such as BAGEL2 model fold-change distributions using reference essential and nonessential genes, incorporating cross-validation and guide-level adjustments. Accelerating these workflows requires strict fidelity to the underlying computational specification—the BAGEL2 scientific contract—which governs pseudorandom sampling trajectories, density lookup grids, numerical aggregation, and tied-score ranking. We developed RSBagel, a high-performance Rust implementation providing an in-memory computation engine, a command-line interface, and a C application binary interface. Evaluating RSBagel against a pinned Python BAGEL2 baseline, RSBagel achieved exact observable parity on genome-wide A375 screens across both single-guide Cas9 perturbations (Brunello, 76,441 sgRNAs; Spearman $\rho = 1.0$) and four-guide single-target Cas12a multiplex arrays (Inzolia, 38,680 constructs; $\rho = 1.0$), yielding 100% top-100 and top-500 candidate overlap and identical precision-recall evaluations. On the canonical HAP1 benchmark (71,090 guides), RSBagel's native deterministic pseudorandom generator, ChaCha8, produced virtually indistinguishable global rankings ($\rho = 0.99996$) with 97/100 top-100 overlap; minor rank variation occurred strictly at the cutoff due to independent algorithmic draw sequences. Across the three benchmark workloads on an Apple M4 Pro workstation, RSBagel achieved an unweighted average acceleration of 22.13-fold (21.27-fold for HAP1, 20.50-fold for Brunello, and 24.60-fold for Inzolia) and an unweighted average complete-pipeline wall-time reduction of 95.45% across five measured trials per configuration. On an extensive synthetic stress workload (B4, 500,000 observations), accelerated runtime (1.953 s versus 48.316 s) was accompanied by an increased peak resident memory footprint (1,307.375 MiB versus 823.609 MiB), establishing that execution speed and memory scaling represent distinct performance dimensions. These findings show that a compiled implementation can substantially accelerate pooled-screen analysis while preserving the qualified scientific contract of the pinned BAGEL2 workflow.

Introduction

Pooled CRISPR fitness screens measure cellular fitness by tracking guide depletion during cell growth. To distinguish biological depletion from technical noise, classifiers evaluate depletion against empirical reference sets of core essential and nonessential genes [1]. BAGEL2 advances this approach through cross-validation, guide-level adjustments, and optional protein interaction networks [2].

As screening datasets scale to genome-wide libraries and multiplex arrays, computational runtime can become an operational bottleneck. Porting pipelines to compiled, memory-safe

languages like Rust [12] offers substantial speedups. However, in statistical classifiers relying on iterative resampling and density estimation, reproducing mathematical equations alone does not ensure reproducible scientific outputs. Numerical stability and downstream biological conclusions depend on the complete computational specification, which we term the "scientific contract." This contract comprises pseudorandom sampling trajectories, cross-validation partitioning, density lookup grids, score aggregation orders, tie-breaking conventions, and output precision. Subtle discrepancies at any stage can shift boundary gene ranks and alter reported essential genes, breaking continuity.

Here, we present RSBagel, a high-performance Rust implementation engineered to preserve the complete BAGEL2 scientific contract while accelerating end-to-end screen execution. We evaluated RSBagel against a pinned Python BAGEL2 baseline across contrasting screening paradigms. We led our validation with two external genome-wide screens in A375 cells: the Brunello screen, which measures individual single-guide RNAs (sgRNAs) in a SpCas9 library [5], and the Inzolia screen, which measures four-guide single-target array constructs in an enCas12a multiplex library [6, 7]. We next evaluated native deterministic pseudorandom sampling using the ChaCha8 algorithm on the canonical HAP1 screen fixture [3]. Finally, we benchmarked complete-pipeline wall-time performance, memory utilization, and API interoperability across real and synthetic stress workloads. Our results demonstrate that RSBagel delivers substantial computational acceleration while preserving exact observable parity on external screens, establishing a reliable, high-throughput foundation for CRISPR fitness analysis.

Results

Defining the BAGEL2 scientific contract and its operational necessity

The BAGEL2 scientific contract defines the complete set of observable computational behaviors necessary to reproduce published screening results identically. This contract encompasses five interconnected operations: (i) pseudorandom sampling order and draw consumption during cross-validation or bootstrap resampling; (ii) serialization precision and numerical formatting at stage boundaries; (iii) Gaussian kernel density discretization and lookup tables; (iv) ordered numerical aggregation of observation-level scores into gene-level summaries; and (v) deterministic ranking and tie-breaking conventions.

Preserving this full contract is essential for reproducible functional genomics. In iterative classifiers like BAGEL2, score distributions are constructed by fitting empirical reference models and scoring held-out observations across cross-validation folds. Minor implementation differences—such as altering floating-point summation order, consuming random draws in a different order, or rounding intermediate fold changes before density evaluation—propagate through cross-validation partitions and alter downstream gene ranks. Upholding the full computational contract ensures that computational acceleration preserves the exact scientific conclusions drawn from the underlying experimental data.

RSBagel is structured to fulfill this contract without sacrificing performance (Figure 1). Organized into three modular Rust crates, the software separates a core in-memory calculation engine from a command-line interface (`rsbagel`) and a C application binary interface (ABI). The computational pipeline executes the canonical BAGEL2 stages: fold-change calculation (`fc`), Bayes-factor estimation (`bf`), and precision-recall analysis (`pr`). Intermediate in-memory bridges replicate the exact serialization boundaries produced by legacy file I/O, allowing developers to execute end-to-end analyses entirely in memory without writing intermediate files to disk while retaining identical numerical outcomes.

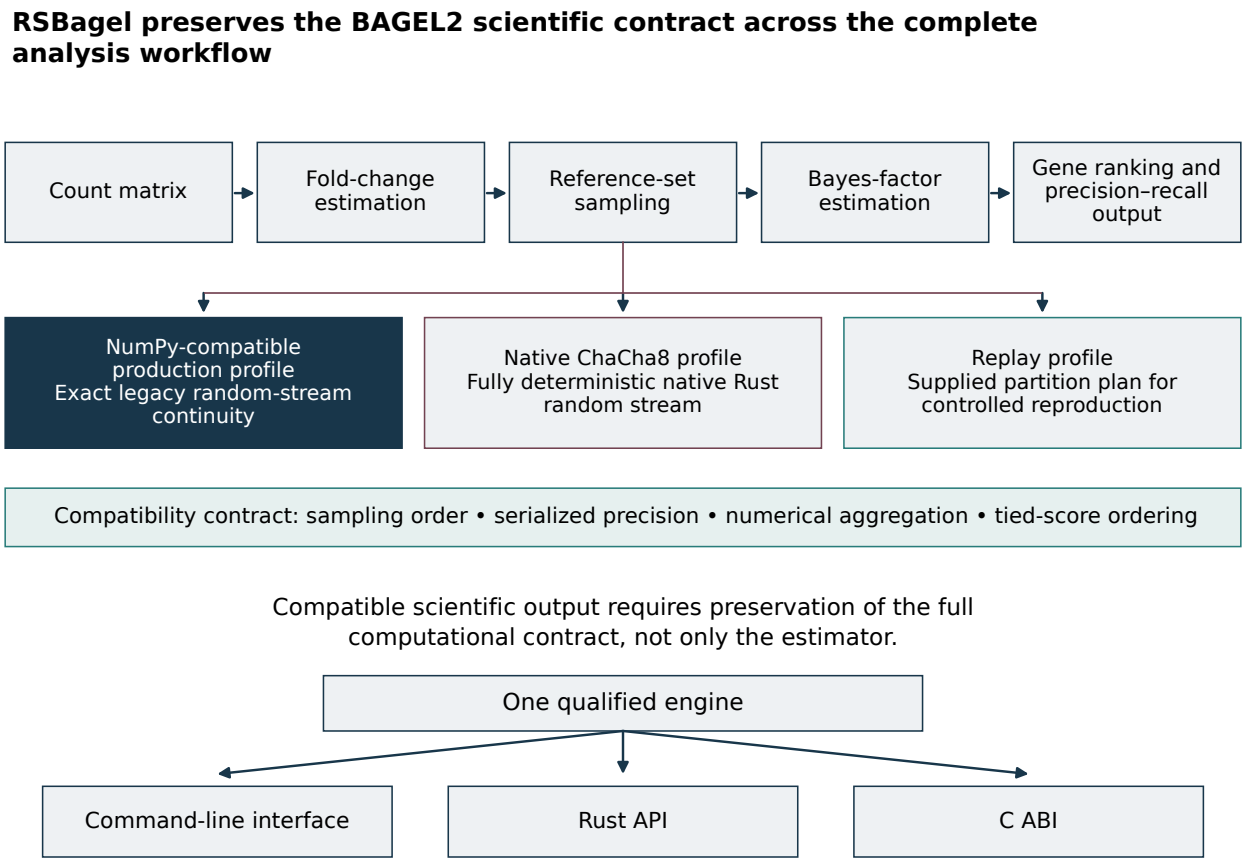


Figure 1. RSBagel preserves the BAGEL2 scientific contract across the complete analysis workflow. The manuscript-defined scientific contract covers observable sampling order, serialized stage-boundary precision, numerical aggregation and tied-score ordering. NumPy-compatible production sampling provides exact legacy stream continuity; native ChaCha8 and replay have separate roles. All interfaces share the qualified engine. Diagram dimensions do not encode runtime, and compatibility does not establish universal biological truth.

RSBagel achieves exact observable parity across external Cas9 and Cas12a screens

To evaluate scientific fidelity on external experimental datasets, we applied RSBagel and legacy Python BAGEL2 to two genome-wide screens in A375 melanoma cells (Table 1). These screens represent distinct functional genomics modalities with contrasting observation units. The Brunello screen [5] employs an SpCas9 library targeting 19,113 genes with 76,441 individual sgRNAs, measuring depletion between plasmid DNA and two replicate day-endpoint samples. In contrast, the Inzolia screen [6, 7] employs an enCas12a multiplex library targeting 19,341 genes with 38,680 four-guide single-target array constructs, measuring depletion between plasmid abundance and day-21 endpoints. Each admitted Inzolia construct is evaluated as a single coherent observation unit representing the intact four-spacer array, without artificially splitting construct counts into guide components.

Table 1. Datasets, observation units, and validation roles

Dataset	Observation unit	Admitted / genes	Baseline → endpoint	Validation role
Brunello A375	One sgRNA	76,441 / 19,113	Original-tracrRNA pDNA → RepA/RepB	External Cas9 parity; timing
Inzolia A375	One four-guide single-target array	38,680 / 19,341	Plasmid_Inzolia → A375_T21A/T21B	External Cas12a parity; timing
HAP1	One guide	71,090 / 18,056	HAP1_T0 → HAP1_T18A/B/C	Native rank stability; production timing

Inzolia counts remain one observation per four-guide single-target array, without guide-count splitting. The two external studies have distinct observed units and do not establish equivalent biological effects.

Under the production NumPy-compatible profile, RSBagel achieved exact observable parity with legacy BAGEL2 across both screens (Figure 2A). Emitted gene scores yielded complete rank concordance (Spearman ρ = 1.0) and 100% candidate overlap across both top-100 (100/100) and top-500 (500/500) sets, with identical precision-recall curves. Furthermore, serialized fold changes, normalized read counts, Bayes factors, and precision-recall files matched character for character.

Exact observable parity describes complete agreement in serialized outputs and ordinal ranks, not universal bitwise register identity. Intermediate values satisfied strict tolerances, with maximum raw score differences of 3.617×10^{-11} for Brunello and 1.140×10^{-11} for Inzolia (Supplementary Methods).

Native ChaCha8 sampling preserves global gene rankings while maintaining determinism

While NumPy-compatible sampling ensures legacy stream continuity, RSBagel also provides a native pseudorandom generator using ChaCha8 [14]. ChaCha8 applies eight cryptographic rounds to yield deterministic sequences for any fixed seed. Because ChaCha8 and NumPy's Mersenne Twister use distinct algorithms, a shared seed (seed 42) produces different random draws and partitions.

Evaluating ChaCha8 on the benchmark HAP1 screen (71,090 guides, 18,056 genes; Table 1) under 1,000 bootstrap iterations produced global gene rankings virtually indistinguishable from legacy BAGEL2 (Spearman ρ = 0.99996; unrounded ρ = 0.999958347522601; Figure 2A). Top-100 candidate overlap reached 97/100, with the three discordant genes located immediately at the boundary (ranks 98–103) due to independent random draw sequences. ChaCha8 is thus a valid, deterministic native profile rather than a failed compatibility mode. The NumPy-compatible profile remains available for historical continuity, while ChaCha8 provides an efficient native alternative.

RSBagel accelerates complete-pipeline execution across diverse screens

Complete-pipeline wall time was benchmarked across five measured trials per group on an Apple M4 Pro workstation (Figure 2B), spanning sequential execution of `fc`, `bf`, and `pr` stages including startup, parsing, statistical modeling, and disk I/O. RSBagel delivered consistent order-of-magnitude accelerations across all workloads: median wall time decreased from 7.928 s to 0.373 s for HAP1 (21.27-fold speedup, 95.30% reduction), from 7.093 s to 0.346 s for Brunello (20.50-fold speedup, 95.12% reduction), and from 5.129 s to 0.208 s for Inzolia (24.60-fold speedup, 95.94% reduction), with tight observed min–max ranges.

Across the three qualified workloads, RSBagel achieved an unweighted average acceleration of 22.13-fold (full-precision mean: 22.125462399912674) and an unweighted average wall-time reduction of 95.45% (full-precision mean: 95.4522045452995%). These metrics represent

unweighted arithmetic means across the three distinct workloads, avoiding artificial pooling or runtime weighting. Automated worker selection utilized the eight hardware performance cores, reducing elapsed time while slightly increasing total CPU work over scalar execution (Supplementary Table S1).

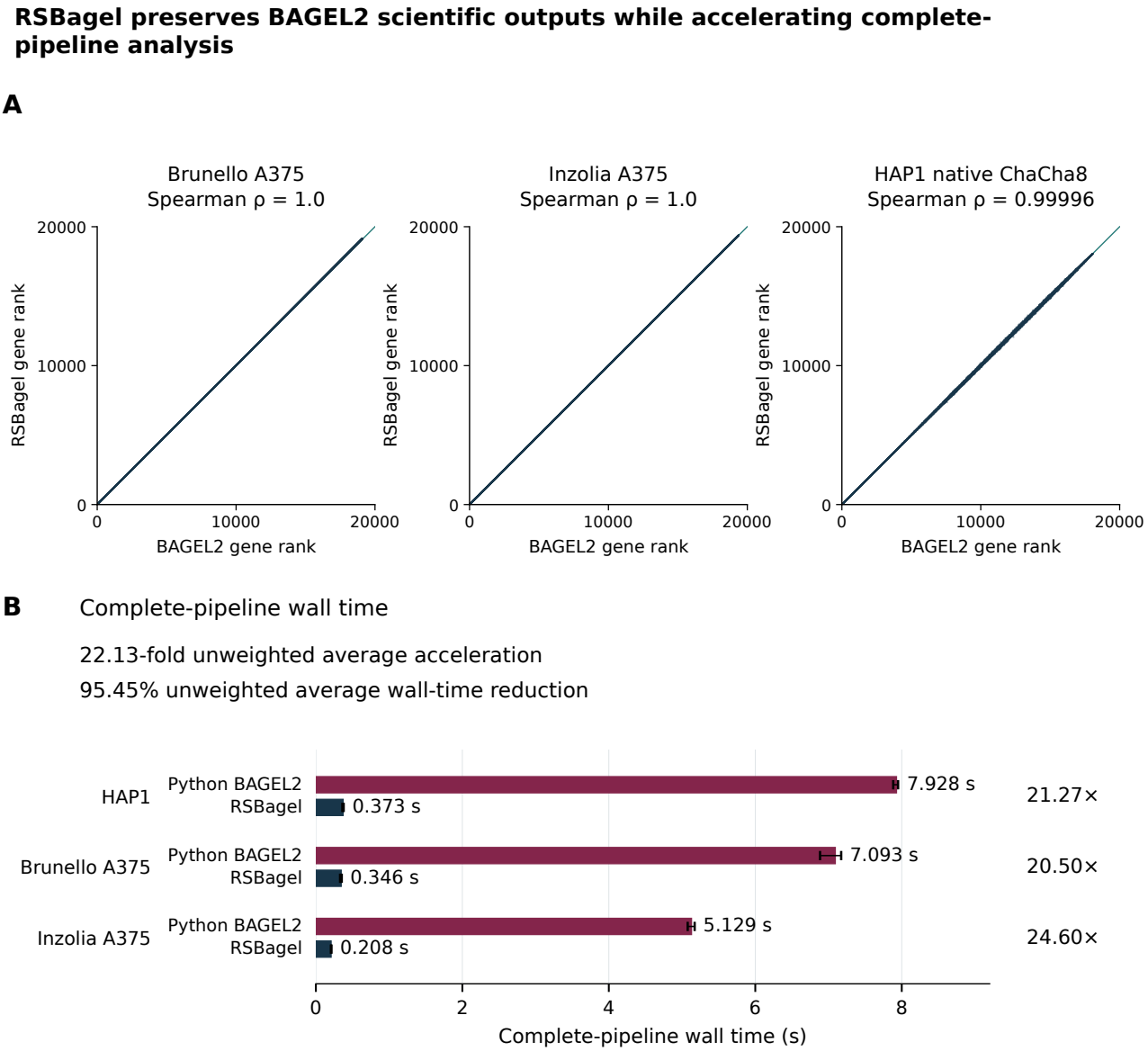


Figure 2. RSBagel preserves BAGEL2 scientific outputs while accelerating complete-pipeline analysis. A, Gene ranks from the qualified NumPy-compatible Brunello and Inzolia workflows and the deterministic native ChaCha8 HAP1 comparison. External A375 workflows show exact observable parity, including serialized scientific outputs and precision-recall ordering; this does not establish universal internal bitwise identity. Native HAP1 has 97/100 top-100 overlap, with minor differences near the top-100 boundary. B, Complete-pipeline median wall times; whiskers span the observed minimum–maximum of five measured runs per group. Ratios use unrounded qualified medians. The unweighted summaries are 22.13-fold average acceleration and 95.45% average wall-time reduction. Workloads are neither pooled nor runtime-weighted. Results apply to the evaluated Apple M4 Pro configurations.

Figure 2 reading guide: A, Brunello A375: ρ = 1.0; Inzolia A375: ρ = 1.0; HAP1 native ChaCha8: ρ = 0.99996. B, Python BAGEL2 → RSBagel median seconds: HAP1, 7.928 → 0.373 (21.27-fold); Brunello A375, 7.093 → 0.346 (20.50-fold); Inzolia A375, 5.129 → 0.208 (24.60-fold). Positions and calculations use qualified unrounded records.

Execution speed and memory utilization represent separate performance dimensions

To assess performance under high data volumes, we benchmarked a large synthetic screening workload (B4) containing 500,000 guide-level observations across 125,000 genes with two experimental replicates. RSBagel reduced median complete-pipeline wall time from 48.316 s in Python BAGEL2 to 1.953 s (observed range 1.919–2.243 s; Supplementary Table S1), demonstrating robust multi-threaded scaling on extensive datasets.

However, computational acceleration did not translate into a universal reduction in memory footprint (Figure S2). For the B4 workload, Python BAGEL2 recorded a median peak resident set size (RSS) of 823.609 MiB, whereas eight-worker RSBagel consumed 1,307.375 MiB—representing an approximately 1.59-fold increase in peak resident memory (ratio 1.56–1.59 across worker configurations). The increase was observed under the qualified eight-worker configuration and shows that faster execution did not guarantee lower peak memory use. In contrast, on the real-data screens (HAP1, Brunello, and Inzolia), RSBagel maintained a lower peak RSS than Python (Supplementary Table S1). These findings illustrate that execution speed and peak memory consumption are distinct dimensions of software performance that must be evaluated independently.

On small workloads, such as the B1 synthetic fixture (1,000 observations across 250 genes), scaling across worker counts was non-monotonic at the millisecond scale, reflecting thread synchronization overhead relative to the trivial computational load (Supplementary Table S1). Consequently, multi-threading should be deployed with consideration of input scale.

Interface flexibility, network modeling, and release boundaries

To support integration into diverse bioinformatics ecosystems, RSBagel provides three complementary interfaces: a standalone command-line binary (`rsbagel`), a reusable in-memory Rust library, and a C application binary interface (ABI). The C ABI exposes flat buffer interfaces and owned handles, incorporating defensive pointer and alignment checks, thread-safe cancellation, and `catch_unwind` panic boundaries that convert unexpected runtime panics into typed error codes (`RSB_INTERNAL_ERROR`). In-memory pipelines allow downstream callers—such as R or Python bindings or interactive visualization tools—to bypass intermediate disk serialization entirely while preserving exact output parity.

BAGEL2 supports optional network-guided essentiality scoring, which integrates prior protein-protein interaction networks into Bayes-factor estimation. In evaluating the legacy codebase, we identified several structural quirks in its network routine: the implementation constructs a hardcoded ten-fold cross-validation plan that ignores user-specified fold settings or no-resampling flags, and guide equalization conflates raw and normalized boosted values within a single list. To ensure complete backward compatibility, RSBagel preserves these legacy behaviors in its default network profile. In addition, RSBagel provides an explicit, opt-in corrected network profile (Figure S1) that reuses the main cross-validation partitions, separates raw and normalized boosted scores, and properly assigns a zero contribution only when an individual gene is absent from a valid fitted model. We emphasize that this corrected profile represents a bounded software contract improvement; its biological efficacy has not been evaluated.

All interface and network evaluations remain strictly bounded to the qualified test fixtures (Supplementary Methods). Diagnostic presentations (such as Python traceback frames versus typed Rust error codes) necessarily differ between languages, and legacy-defect compatibility remains distinct from endorsing the scientific validity of legacy software flaws.

Discussion

Re-engineering scientific software in modern compiled languages offers substantial opportunities to improve computational efficiency, but speedup must not come at the expense of scientific reproducibility. In this study, we demonstrated that RSBagel preserves the complete BAGEL2 scientific contract while accelerating pooled CRISPR fitness-screen analysis by an average of 22.13-fold (95.45% average wall-time reduction) across diverse experimental screens on Apple Silicon.

Our findings underscore why contract preservation is fundamental in computational biology. Replicating the theoretical equations of a statistical model is often insufficient to reproduce published results. In iterative classifiers such as BAGEL2, score distributions are sensitive to implementation details: the exact pseudorandom sequence governing data partitioning, the floating-point discretization of kernel density grids, the accumulation order of guide log-ratios, and the resolution of tied scores. Subtle discrepancies at any stage propagate through cross-validation folds, shifting boundary gene scores and altering essentiality classifications. By capturing and formalizing these behaviors into an explicit scientific contract, RSBagel ensures that computational acceleration preserves the exact scientific conclusions drawn from the underlying screening data.

The evaluation of RSBagel across external A375 screens highlights the practical breadth of this contract. Screen libraries differ fundamentally in their targeting mechanisms and data representations. The Brunello library represents conventional single-guide Cas9 screens, where each sgRNA is an independent observation. In contrast, the Inzolia library represents multiplex Cas12a screening, where each construct contains a four-guide single-target array. By evaluating each Inzolia array as an intact observation unit, RSBagel achieved exact observable rank parity ($\rho = 1.0$) and 100% top-candidate concordance across both Cas9 and Cas12a modalities. This confirms software fidelity across distinct experimental paradigms without implying biological equivalence between nucleases or library architectures.

Our investigation of pseudorandom number generators clarifies the distinction between determinism and legacy continuity. ChaCha8 provides a deterministic native pseudorandom-number generator when initialized with a fixed seed. On the HAP1 benchmark, ChaCha8 produced a global gene ranking virtually indistinguishable from Python BAGEL2 ($\rho = 0.99996$), with 97/100 top-100 overlap. The three divergent genes were located directly at the top-100 boundary, illustrating how independent random sampling trajectories introduce minor stochastic variation near cutoffs. Rather than representing an implementation flaw, ChaCha8 serves as a valid native profile for independent pipelines. Concurrently, RSBagel's NumPy-compatible profile ensures exact draw-for-draw reproduction when legacy continuity is required.

The synthetic benchmark results emphasize that computational speed and memory utilization are independent dimensions. While RSBagel accelerated the large B4 workload from 48.316 s to 1.953 s, its peak resident memory increased by 1.59-fold (1,307.375 MiB versus 823.609 MiB). The increase was observed under the qualified eight-worker configuration and shows that faster execution did not guarantee lower peak memory use. Conversely, on real-data workloads, RSBagel utilized less peak memory than Python. These observations demonstrate that multi-threaded compiled engines require conscious memory profiling when deployed at scale.

Limitations

Several limitations define the boundaries of this study. First, software agreement was evaluated against a pinned BAGEL2 source commit (build 115, commit `53388adbb4fb0931e5c9dda135502be19e4555f0`) under specific Python and Rust compiler versions. Performance measurements reflect an active-desktop Apple M4 Pro workstation under macOS; relative acceleration, memory footprints, and multi-core scaling may vary across different CPU architectures (such as x86_64 server platforms), operating systems, and memory hierarchies.

Second, dataset curation followed published study annotations. Inzolia array classifications relied on author-provided metadata without independent genomic alignment or off-target re-evaluation, and plasmid-to-endpoint depletion does not reconstruct an unobserved cellular baseline. Precision-recall metrics were evaluated against standard reference gene sets (CEGv2 and NEGv1 [4]), which serve as benchmark standards rather than absolute biological ground truth.

Third, while RSBagel provides a corrected network profile that resolves structural flaws in legacy network modeling, its validation is restricted to software contract behavior; no biological advantage has been empirically demonstrated. Finally, this package represents an evaluated scientific implementation rather than a commercially signed, notarized, or publicly distributed software package.

Practical value and boundaries

The command-line interface, Rust library, and C ABI provide several routes for integration into screening and bioinformatics workflows.

Methods

RSBagel architecture and interface design

RSBagel is implemented in Rust 2021 edition and structured into three modular crates: a core computational engine (`rsbagel-core`), a command-line interface (`rsbagel-cli`), and a C-compatible application binary interface (`rsbagel-capi`) (Figure 1). Scientific computations use double-precision (64-bit) floating-point arithmetic throughout, avoiding Python interpreter dependencies during production execution.

The command-line interface exposes three sequential subcommands corresponding to canonical BAGEL2 stages: fold-change calculation (`fc`), Bayes-factor estimation (`bf`), and precision-recall evaluation (`pr`). To enable integration into foreign language environments (such as C, C++, Python, or R) and workflow managers, the C ABI exposes flat memory buffer interfaces and owned opaque handles. The C ABI performs defensive boundary validation—checking pointer nullity, memory alignment, buffer lengths, and integer overflow conditions—and provides cooperative cancellation and progress callback mechanisms. To prevent panics from crossing the foreign function boundary, top-level C API functions wrap internal execution in `std::panic::catch_unwind`, converting any Rust panic into a structured error code (`RSB_INTERNAL_ERROR`).

To support pure-memory execution, the engine incorporates precision and ordering bridges that reproduce the exact scientific boundaries created when the legacy CLI serializes intermediate tables to disk. Specifically, the fold-change bridge truncates normalized counts to two decimal places and fold changes to three decimal places before Bayes-factor estimation, while the Bayes-factor bridge rounds emitted scores to three decimal places and enforces alphabetical gene ordering before precision-recall analysis. This design enables in-memory pipelines to achieve exact observable parity with file-based workflows without incurring disk I/O overhead.

BAGEL2 scientific-contract specifications

The comparator for all parity evaluations was unmodified BAGEL2 version 2.0, build 115 (commit `53388adbb4fb0931e5c9dda135502be19e4555f0`) [8]. The reference Python environment was frozen using Python 3.11.16, NumPy 1.23.5, SciPy 1.10.1, pandas 1.5.3, scikit-learn 1.2.2, and Click 7.1.2. Rust qualification utilized Rust 1.98.0 and a locked Cargo dependency manifest.

The fold-change stage adds a pseudocount (default $p = 5$) to raw read counts, computes library-size normalization relative to 10^7 reads per sample, filters guides below a minimum control read threshold, and calculates log2 fold changes between endpoint samples and controls (Equations 1–3 in Supplementary Methods). The Bayes-factor stage fits Gaussian kernel density estimates (KDE) to reference essential and nonessential gene fold changes using Scott's rule for bandwidth selection [13]. RSBagel replicates the exact support bounds, quantized lookup grid (step size 0.01), and linear extrapolation models of the legacy implementation. Observation-level Bayes factors are summed across replicates and guides to produce gene-level summaries. Standard deviations are calculated using the population convention (degrees of freedom = 0). The precision-recall stage orders emitted gene scores and calculates recall, precision, and false discovery rate (FDR) using observed reference genes as the denominator, adhering to legacy pandas sorting conventions for tied scores.

Pseudorandom-number profiles and worker allocation

RSBagel implements two distinct pseudorandom number generator (PRNG) profiles: a NumPy-compatible profile and a native ChaCha8 profile. The NumPy-compatible profile faithfully replicates the legacy NumPy RandomState Mersenne Twister implementation [11], including draw sequences, ordered train/test partitions, and stream continuation across cross-validation folds. A replay mode is also provided, allowing users to supply an existing partition plan directly to isolate numerical calculation from sampling variance.

The native profile utilizes the ChaCha8 algorithm with eight rounds of cryptographic mixing [14]. ChaCha8 is completely deterministic when initialized with a fixed seed and stream identifier. However, because its algorithmic formulation differs from the Mersenne Twister, identical numerical seeds produce distinct pseudorandom sequences between ChaCha8 and NumPy.

Multi-threaded parallel execution in RSBagel utilizes Rayon. To preserve determinism across thread counts, parallel loops employ ordered reductions and deterministic chunk partitioning. Automatic worker allocation detects the hardware performance-core count on Apple Silicon (capped at eight). If hardware topology is unavailable, it defaults to available system parallelism (capped at eight, minimum one). Explicit worker counts override automatic selection.

Datasets, observation units, and curation

We evaluated RSBagel across three real screening datasets (Table 1).

Brunello A375 screen

Raw read counts and sgRNA annotations were obtained from Supplementary Data 1 of Sanson et al. [5]. We selected original-tracrRNA plasmid DNA counts as baseline and A375 RepA/RepB counts as endpoints. Exact sequence-to-gene matching retained 76,441 sgRNAs across 19,113 genes, covering 670 of 684 essential (CEGv2) and 903 of 927 nonessential (NEGv1) reference genes [4]. We excluded 1,000 non-targeting control annotations prefixed with `NO_CURRENT_`. The observation unit is one individual sgRNA.

Inzolia A375 screen

Design and count records were obtained from Supplementary Data 5 and 6 of Esmaeili Anvar et al. and the versioned Figshare repository [6, 7]. We selected one-component plasmid counts as baseline and A375 day-21 counts (T21A/T21B) as endpoints. Filtering required the four-spacer direct-repeat architecture, unambiguous gene targeting, and complete count records, retaining 38,680 construct-level observations across 19,341 genes, covering 664 essential and 864 nonessential reference genes. We excluded mixed-target arrays, control arrays, and unmapped constructs. Each admitted observation represents an intact four-guide single-target array construct without artificial guide count splitting.

HAP1 screen fixture

Obtained from the canonical BAGEL2 distribution [8, 3], comprising 71,090 guide rows across 18,056 genes with three experimental replicates (T18A, T18B, T18C) compared against baseline (T0).

Scientific-agreement and fidelity metrics

Scientific fidelity was assessed across intermediate and final outputs: normalized counts, fold changes, cross-validation partitions, kernel density models, observation Bayes factors, gene summaries, and precision-recall tables. Raw numerical comparisons were evaluated against strict pre-established tolerances: absolute 10^{-12} for normalized counts, 10^{-10} for fold changes, 10^{-10} absolute plus 10^{-8} relative for density estimates, and 10^{-8} for gene summaries.

Serialized output files were evaluated for exact character-for-character equivalence. Gene rank concordance was measured using the Spearman rank correlation coefficient (ρ) [9, 10]. Top-set candidate overlap was calculated as the size of the set intersection divided by the target set size (top 100 or top 500).

Complete-pipeline performance benchmarking

Complete-pipeline performance was benchmarked on an Apple M4 Pro workstation (Mac16,11; eight performance cores, four efficiency cores, 24 GiB unified memory, macOS 27.0.1 arm64). Timing runs measured wall time for fresh, sequential execution of `fc`, `bf`, and `pr` processes from input files to output tables, encompassing process initialization, data parsing, modeling, and I/O. Supervisor overhead and post-run validation were excluded.

Each benchmark configuration comprised one unmeasured warmup run followed by five measured trials. Python numerical libraries (NumPy/OpenBLAS) were restricted to a single thread (`OPENBLAS_NUM_THREADS=1`, `OMP_NUM_THREADS=1`). Benchmarks evaluated Python BAGEL2 against RSBagel using eight workers (explicit eight workers for HAP1 and synthetic workloads; automatic worker selection resolving to eight performance cores for Brunello and Inzolia). All timed trials passed output verification checks.

Average acceleration was computed as the unweighted arithmetic mean of the three workload-specific median speed ratios calculated from full-precision wall times (22.125462399912674, reported as 22.13-fold). Average wall-time reduction was computed as the unweighted arithmetic mean of the three workload-specific percentage reductions calculated from full-precision medians (95.4522045452995%, reported as 95.45%).

Memory measurement protocol

Peak resident set size (RSS) was measured as the maximum high-water mark among child processes across the sequential `fc`, `bf`, and `pr` stages using Darwin OS kernel metrics. Peak memory bytes were divided by 1,048,576 to report MiB. Reported values represent the median across five measured trials per configuration.

Interface and network qualification

Network qualification evaluated default legacy and opt-in corrected profiles against reduced network fixtures (Figure S1). In-memory execution was validated through memory-only pipelines, C11 test harnesses, and Swift header integrations.

Data and code availability

BAGEL2 source code is available from the Hart Lab repository under commit `53388adbb4fb0931e5c9dda135502be19e4555f0` [8]. Brunello and Inzolia screening datasets are available through their original publications and data repositories [5, 6, 7]. The HAP1 fixture and reference gene sets (CEGv2, NEGv1) are available through the BAGEL2 distribution. The RSBagel manuscript source, build automation, vector figures, and benchmark evidence records are maintained within the project-control publication package.

Funding

This research received no external funding.

Competing interests

The author declares no competing interests.

Author contributions

Yoon Sing Yap conceived the study, developed the software and methodology, performed the analyses and validation, interpreted the results, prepared the figures, and wrote and revised the manuscript.

References

1. Hart T, Moffat J (2016). BAGEL: a computational framework for identifying essential genes from pooled library screens. BMC Bioinformatics 17, 164. https://doi.org/10.1186/s12859-016-1015-8

2. Kim E, Hart T (2021). Improved analysis of CRISPR fitness screens and reduced off-target effects with the BAGEL2 gene essentiality classifier. Genome Medicine 13, 2. https://doi.org/10.1186/s13073-020-00809-3

3. Hart T et al. (2017). Evaluation and Design of Genome-Wide CRISPR/SpCas9 Knockout Screens. G3: Genes, Genomes, Genetics 7, 2719-2727. https://doi.org/10.1534/g3.117.041277

4. Hart T, Brown KR, Sircoulomb F, Rottapel R, Moffat J (2014). Measuring error rates in genomic perturbation screens: gold standards for human functional genomics. Molecular Systems Biology 10, 733. https://doi.org/10.15252/msb.20145216

5. Sanson KR, Hanna RE, Hegde M, Donovan KF, Strand C, Sullender ME, Vaimberg EW, Goodale A, Root DE, Piccioni F, Doench JG (2018). Optimized libraries for CRISPR-Cas9 genetic screens with multiple modalities. Nature Communications 9, 5416. https://doi.org/10.1038/s41467-018-07901-8

6. Esmaeili Anvar N, Lin C, Ma X, Wilson LL, Steger R, Sangree AK, Colic M, Wang SH, Doench JG, Hart T (2024). Efficient gene knockout and genetic interaction screening using the in4mer CRISPR/Cas12a multiplex knockout platform. Nature Communications 15, 3577. https://doi.org/10.1038/s41467-024-47795-3

7. Traver Hart, Chenchu Lin, Nazanin Esmaeili Anvar (2024). Efficient gene knockout and genetic interaction screening using the in4mer CRISPR/Cas12a multiplex knockout platform. Figshare version 1. https://doi.org/10.6084/m9.figshare.24243832.v1

8. Hart Lab (accessed 2026). BAGEL2 source code, version 2.0, build 115. https://github.com/hart-lab/bagel/tree/53388adbb4fb0931e5c9dda135502be19e4555f0

9. Spearman C (1904). The Proof and Measurement of Association between Two Things. The American Journal of Psychology 15, 72-101. https://doi.org/10.2307/1412159

10. SciPy community (accessed 2026-11-01). scipy.stats.spearmanr: SciPy v1.10.1 Manual. https://docs.scipy.org/doc/scipy-1.10.1/reference/generated/scipy.stats.spearmanr.html

11. NumPy developers (accessed 2026-11-01). Legacy Random Generation: NumPy v1.23 Manual. https://numpy.org/doc/1.23/reference/random/legacy.html

12. Klabnik S, Nichols C, Krycho C, with contributions from the Rust Community (accessed 2026-11-01). The Rust Programming Language. https://doc.rust-lang.org/book/

13. SciPy community (accessed 2026-11-01). scipy.stats.gaussian_kde: SciPy v1.10.1 Manual. https://docs.scipy.org/doc/scipy-1.10.1/reference/generated/scipy.stats.gaussian_kde.html

14. Rand project developers (accessed 2026-11-01). rand_chacha 0.3.1. Official Rust crate documentation. https://docs.rs/rand_chacha/0.3.1/rand_chacha/

Supplementary Methods

S1. Normalization and stage boundaries

For observation i and sample j, the pipeline adds pseudocount p once to each selected sample and once to the combined control. If r represents a sample read count and c represents the sum of selected control counts, normalized read counts are defined as:

$$n_{ij} = 10^7 \frac{r_{ij} + p}{\sum_k (r_{kj} + p)}$$

Text form (1): n[i,j] = 10^7 × (r[i,j] + p) / (sum over k of (r[k,j] + p)).

$$n_{ic} = 10^7 \frac{c_i + p}{\sum_k (c_k + p)}$$

Text form (2): n[i,c] = 10^7 × (c[i] + p) / (sum over k of (c[k] + p)).

$$FC_{ij} = \log_2\left(\frac{n_{ij}}{n_{ic}}\right)$$

Text form (3): FC[i,j] = log2(n[i,j] / n[i,c]).

Sample totals sum across all rows prior to filtering. Filtering retains observations where c_i + p meets or exceeds the minimum-read threshold. Log2 fold changes are calculated as the ratio of normalized sample count to normalized control count. The default pseudocount is p = 5. In the legacy pipeline, normalized counts are serialized with two decimal places and fold changes with three decimal places. The in-memory fold-change-to-Bayes-factor bridge faithfully reproduces this boundary precision.

The Bayes-factor stage retains the legacy double-precision Gaussian kernel density model, Scott's rule for bandwidth selection, support bounds, hundredth-grid lookup discretization, and regression-based score calibration. Gene observation summaries follow legacy accumulation order. Standard deviations are computed using the population convention (degrees of freedom = 0). For each cross-validation or bootstrap iteration, a gene's score represents the ordered sum of

its constituent observation and replicate contributions; standard deviations summarize the variation across iteration sums rather than summing individual observation variances.

The Bayes-factor-to-precision-recall bridge rounds emitted scores to three decimal places, enforces alphabetical ordering across input rows, and adheres to pandas stable-sort tie-breaking conventions. Precision-recall curves evaluate all sorted rows, including unlabeled genes. If reference gene lists overlap, essential annotations take precedence. Recall is computed using the number of observed essential genes as the denominator. Precision is defined as TP / (TP + FP) once an annotated gene occurs, defaulting to 1.0 prior to the first labeled gene. False discovery rate (FDR) is defined as 1 - precision. No artificial initial curve points or tied-score groupings are introduced.

S2. Dataset projection and exclusions

For the Brunello screen, raw counts and sgRNA annotations were derived from Supplementary Data 1 of Sanson et al. [5]. Columns B, C, and D supplied original-tracrRNA plasmid DNA baseline, RepA, and RepB endpoint counts, respectively. Exact sequence-to-gene joins preserved original row order. Observations from the modified-tracrRNA condition were excluded. A total of 1,000 non-targeting control annotations prefixed with `NO_CURRENT_` were recorded in the exclusion inventory rather than evaluated as gene targets.

For the Inzolia screen, library design and count records were obtained from Supplementary Data 5 and 6 of Esmaeili Anvar et al. and the versioned Figshare repository [6, 7]. Joining by construct identifier selected columns I, M, and N: Plasmid_Inzolia baseline and A375 day-21 replicates T21A and T21B. Original construct identities, sequences, four spacer sequences and relative positions, and gene annotations were preserved. Admission required the four-spacer direct-repeat architecture and a single unambiguous gene target.

Across all 51,921 Inzolia count records, constructs were categorized as follows: 38,976 singleton arrays (of which 296 lacked complete one-component count data); 9,794 mixed-target arrays; 570 control arrays; and 2,581 ambiguous or unmapped arrays. This classification yielded 38,680 admitted construct observations and 13,241 excluded records. Control constructs included EGFP, intergenic, and non-targeting arrays. Other cell lines, two-component assays, and drug-treated conditions were excluded. Author annotations were preserved without independent genomic re-alignment. The observed plasmid baseline was utilized as the experimental reference.

The HAP1 dataset serves as a benchmark software fixture bundled with the canonical BAGEL2 release, supplying 71,090 guide rows across 18,056 genes and three experimental replicates. The software evaluation benchmarks computational behavior against this frozen fixture rather than reconstructing the historical wet-lab screen.

S3. Qualification categories

Software qualification evaluated production parity, legacy-defect compatibility, expected failure handling, corrected-profile behavior, and native random sampling separately. Numerical tolerances remained strict: absolute 10^{-12} for normalized read counts; absolute 10^{-10} for fold changes; absolute 10^{-10} plus relative 10^{-8} for density calculations; absolute 10^{-9} plus relative 10^{-8} for fitted models; and absolute 10^{-8} for gene scores and precision-recall metrics. Emitted scientific text files were evaluated for exact character identity independently of numerical tolerances.

Production qualification verified 48 CLI configurations across 20,831 raw numerical comparisons, accompanied by 32 automated workspace test suites. Minor diagnostic discrepancies—such as Python traceback formatting, numerical warning displays, and error

message phrasing—reflect differences between Python runtime exceptions and structured Rust error types. Successful pipeline executions, required partial outputs, exit codes, and output ordering were verified against established contracts.

In pure-memory HAP1 bootstrap evaluations under NumPy-compatible sampling, RSBagel matched all 1,000 ordered partition plans identically. Maximum absolute differences between Python and RSBagel across all iterations were 1.876×10^{-12} for gene score means and 1.583×10^{-12} for standard deviations. Under native ChaCha8 sampling on HAP1, the maximum absolute mean difference was 3.545 and maximum rank difference was 306, reflecting independent algorithmic draw sequences.

Across all 312 external raw comparison records for the A375 screens, intermediate values passed all numerical tolerances without exception. Maximum absolute differences in raw gene summary scores were 3.617×10^{-11} for Brunello and 1.140×10^{-11} for Inzolia.

S4. Timing and source identities

The original timing benchmark suite utilized commit `189964eb78f09ad4e82c783600efb0a938803378`, evaluating explicit worker configurations of one, two, four, and eight threads. The final A375 external validation suite utilized commit `669cba807498399b3a6f173d69ecfc6f78891920`, with automatic worker selection resolving to eight performance cores.

The HAP1 and synthetic measurement series comprised 125 measured pipeline executions and 25 warmup runs. The A375 measurement series comprised 30 measured pipeline executions and 6 warmup runs. All 186 pipeline executions completed successfully and passed output validation checks; no formal trial was replaced.

The legacy comparator was BAGEL2 build 115 at commit `53388adbb4fb0931e5c9dda135502be19e4555f0` (entry-point SHA-256 `89f9ba4115513caf9838ce40ab4ca8d47e5e2bc82cd128f356185997ddf9a6c7`). Python 3.11.16 was configured with NumPy 1.23.5, SciPy 1.10.1, pandas 1.5.3, scikit-learn 1.2.2, and Click 7.1.2, with numerical library threads fixed at one. Rust qualification used Rust 1.98.0 / LLVM 22.1.8 and locked dependencies.

Wall time measured the sequential execution of fresh `fc`, `bf`, and `pr` processes, including process initialization, input parsing, computation, and file emission. Supervisor startup and output validation were excluded from timed intervals. CPU time represents each trial's cumulative user and system time summed across child processes. Peak RSS represents the maximum child process high-water mark. Benchmarks were conducted on an active-desktop workstation without artificial thread affinity pinning.

S5. Synthetic workloads

Synthetic screening fixtures evaluated scaling across problem sizes: B1 (1,000 observations, 250 genes, two replicates), B3 (80,000 observations, 20,000 genes, strong depletion), and B4 (500,000 observations, 125,000 genes, moderate depletion). Each synthetic gene comprised four observations. Read counts were synthesized with deterministic pseudorandom perturbations, with nominal reference fractions of 4% essential and 5% nonessential genes. B1 bootstrap evaluated 1,000 resampling iterations. All synthetic benchmarks utilized identical input files and seed 42.

Supplementary Results

S6. Complete performance distributions

Supplementary Table S1 presents a selected summary of Python BAGEL2 and RSBagel performance across real and synthetic workloads. The complete dataset of all 31 measured worker-profile groups is preserved in `data/Table-S1-formal-summary.csv`, recording wall time, CPU time, peak RSS, campaign identifiers, worker modes, repetition counts, and source commit hashes. Individual trial measurements for all 155 measured runs are preserved in `data/Table-S2-measured-trials.csv`.

On the small B1 cross-validation workload, scaling across worker counts was non-monotonic at millisecond scale due to thread management overhead. On B1 bootstrap (1,000 iterations), RSBagel recorded a median wall time of 0.133 s compared with 15.773 s for Python. On the extensive B4 workload, RSBagel accelerated runtime from 48.316 s to 1.953 s while increasing peak resident memory from 823.609 MiB to 1,307.375 MiB (Figure S2). Reported summary metrics reflect unweighted arithmetic means across workloads rather than pooled or runtime-weighted estimates.

S7. Rank sensitivity and network behavior

On the external A375 screens, RSBagel achieved exact observable rank and precision-recall parity under legacy sorting rules. Diagnostic stable sorting of unrounded raw scores reordered 957 Brunello and 751 Inzolia genes within tied score clusters, demonstrating the importance of the legacy tie-breaking convention in preserving observable outputs. On HAP1, native ChaCha8 preserved global rank order ($\rho = 0.99996$; unrounded 0.999958347522601) and top-100 overlap (97/100), with minor variation restricted to the boundary between independent random streams.

On the reduced network fixture, legacy BAGEL2 with no resampling produced 108 NaN boosted scores across 120 genes with zero exit status, and evaluating 11 folds failed after 10 folds while leaving partial files on disk. RSBagel's default network profile preserves these quirks for backward compatibility. The opt-in corrected profile reuses main-stage partitions, treats missing valid-model gene contributions as zero, returns structured errors for invalid models, and maintains separate raw and normalized boosted scores (Figure S1).

Supplementary Tables

Table S1. Selected view of complete-pipeline time and memory

Workload / type	Engine	Wall time (s)	CPU time (s)	Peak RSS (MiB)
Real · Brunello CV	Python BAGEL2	7.093 [6.884–7.173]	6.931 [6.739–6.945]	217.547 [214.328–222.547]
Real · Brunello CV	RSBagel	0.346 [0.331–0.353]	0.509 [0.501–0.518]	196.703 [195.812–197.609]
Real · Inzolia CV	Python BAGEL2	5.129 [5.077–5.172]	4.988 [4.905–5.018]	181.906 [180.734–183.938]
Real · Inzolia CV	RSBagel	0.208 [0.205–0.213]	0.279 [0.275–0.282]	126.219 [124.156–126.906]
Real · HAP1 CV	Python BAGEL2	7.928 [7.884–7.950]	7.823 [7.787–7.836]	224.328 [224.000–227.531]
Real · HAP1 CV	RSBagel	0.373 [0.362–0.379]	0.608 [0.602–0.609]	205.125 [202.062–207.688]
Synthetic B1 CV	Python BAGEL2	2.221 [2.191–2.263]	2.124 [2.106–2.163]	125.578 [125.516–125.828]
Synthetic B1 CV	RSBagel	0.042 [0.027–0.045]	0.017 [0.016–0.017]	6.312 [6.234–6.562]
Synthetic B3 CV	Python BAGEL2	7.657 [7.615–7.964]	7.537 [7.499–7.743]	217.562 [216.203–219.047]
Synthetic B3 CV	RSBagel	0.340 [0.332–0.346]	0.554 [0.550–0.573]	206.031 [202.812–216.219]
Synthetic B4 CV	Python BAGEL2	48.316 [48.058–48.711]	48.108 [47.863–48.270]	823.609 [821.188–827.641]
Synthetic B4 CV	RSBagel	1.953 [1.919–2.243]	3.446 [3.407–3.547]	1,307.375 [1,305.359–1,313.438]
Synthetic B1 bootstrap1000	Python BAGEL2	15.773 [15.469–16.108]	15.548 [15.255–15.711]	198.062 [197.578–198.625]
Synthetic B1 bootstrap1000	RSBagel	0.133 [0.125–0.145]	0.383 [0.372–0.399]	100.422 [100.312–100.609]

Median [observed minimum–maximum], n = 5 measured runs, one warmup excluded. RSBagel uses the qualified eight-worker configuration. Real and synthetic workloads are identified explicitly. RSS is maximum stage child high-water mark. This selected view displays 14 of 31 worker-profile groups. Full precision, all 31 worker-profile groups and all 155 measured trials remain in the included source CSVs.

Table S2. Individual measured trials

The included data/Table-S2-measured-trials.csv retains all 155 measured trial records with wall seconds, CPU seconds, peak RSS bytes, campaign, engine, worker mode and stored array index. Stored array order is not asserted to be chronological.

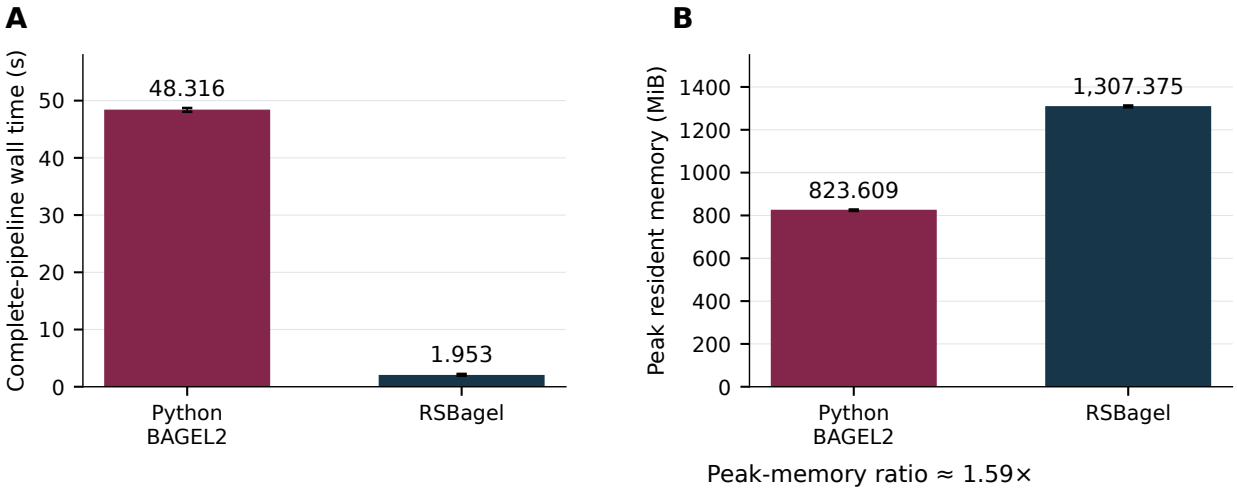
Legacy and corrected network compatibility profiles

Interaction	Legacy compatibility profile	Corrected opt-in profile
Partition plan	Fresh fixed ten-fold network plan	Reuse exact main partitions
No resampling	Preserve source nonfinite behavior	Reuse all-gene main plan
Nondefault folds	Preserve fixed-fold mismatch	Remove fixed-fold mismatch
Guide equalization	Preserve raw/normalized double append	Separate BoostedBF / BoostedNormBF
Missing contribution	Preserve source behavior	Zero per gene from valid model; invalid models remain errors

Legacy behavior is preserved for compatibility.
Correction is explicit and optional. No biological-superiority claim is made.

Figure S1. Legacy and corrected network compatibility profiles. Legacy behavior is preserved for compatibility. Corrected behavior is explicit and optional: it reuses main partitions, separates raw and normalized boosted scores, and distinguishes per-gene missing contributions from invalid models. These are bounded software-contract tests, not a biological-superiority claim.

Faster execution can require more peak memory



RSBagel reduced B4 wall time while using more peak resident memory.
Execution speed and memory use are separate performance dimensions.

Figure S2. Faster execution can require more peak memory. A, B4 complete-pipeline wall time. B, B4 maximum child peak resident memory. Paired bars show Python BAGEL2 and qualified eight-worker RSBagel: medians with descriptive observed min–max whiskers, n = 5 measured runs. Peak RSS is 823.609 and 1,307.375 MiB, respectively (approximately 1.59×). RSBagel reduced B4 wall time while using more peak resident memory. Execution speed and memory use are separate performance dimensions.